Dynamic Decode Attention

Per-step task scheduling that balances KV work across CTAs

Tencent Hunyuan AI Infra · HPC-Ops

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Attention Kernels · github.com/Tencent/hpc-ops

Decode batches mix 128K and 4K contexts

- Continuous batching puts requests at every stage into one launch
- Long-context, multi-turn and agentic traffic widen KV lengths
- Attention work tracks live KV: 16K is **16×** the work of 1K

Same KV work, 4.3× the latency

4.3×

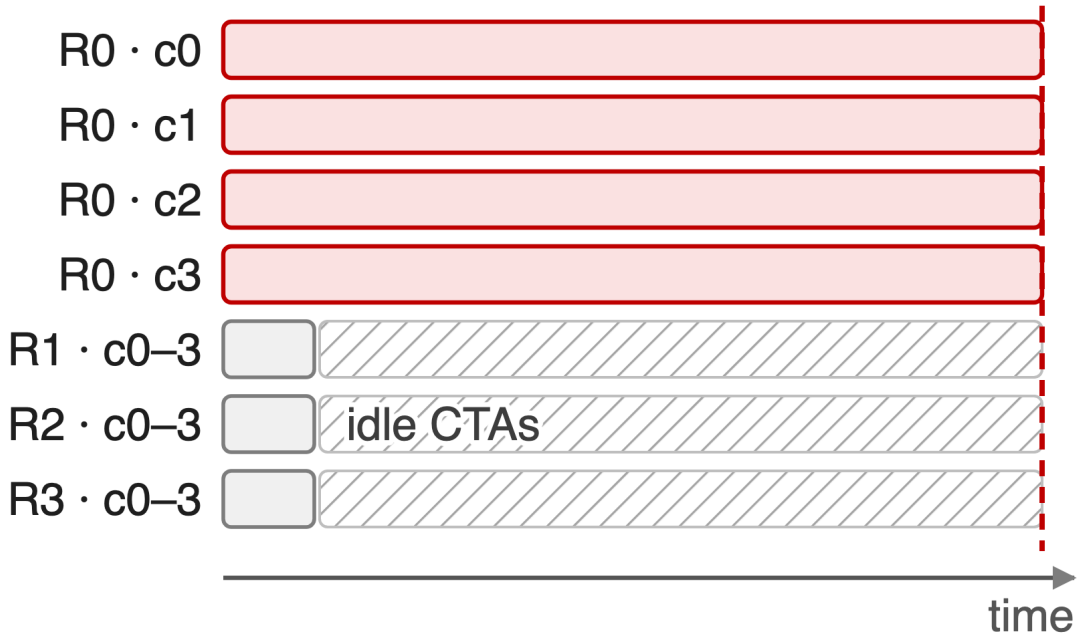
longer static split-KV attention for 1×128K + 31×4K than for 64×4K, with ~2% less KV to read

Derived from LMSYS / Tencent "HPC-Ops × SGLang" Table 1 (H20, FP8 KV): 0.186 vs 0.043 ms; 258,048 vs 262,144 KV tokens

Static split-KV either stalls or wastes CTAs

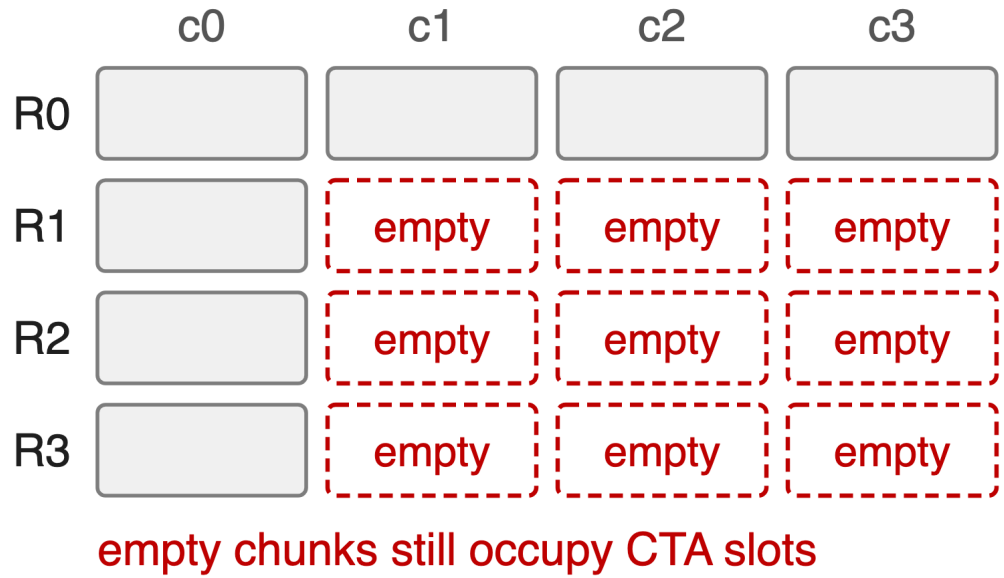
Fixed split count

4 chunks per request



Fixed chunk size

grid sized for the longest request



Launch grid = KV heads × requests × chunks, fixed before the live lengths are seen

Insight: balance KV tiles, not requests

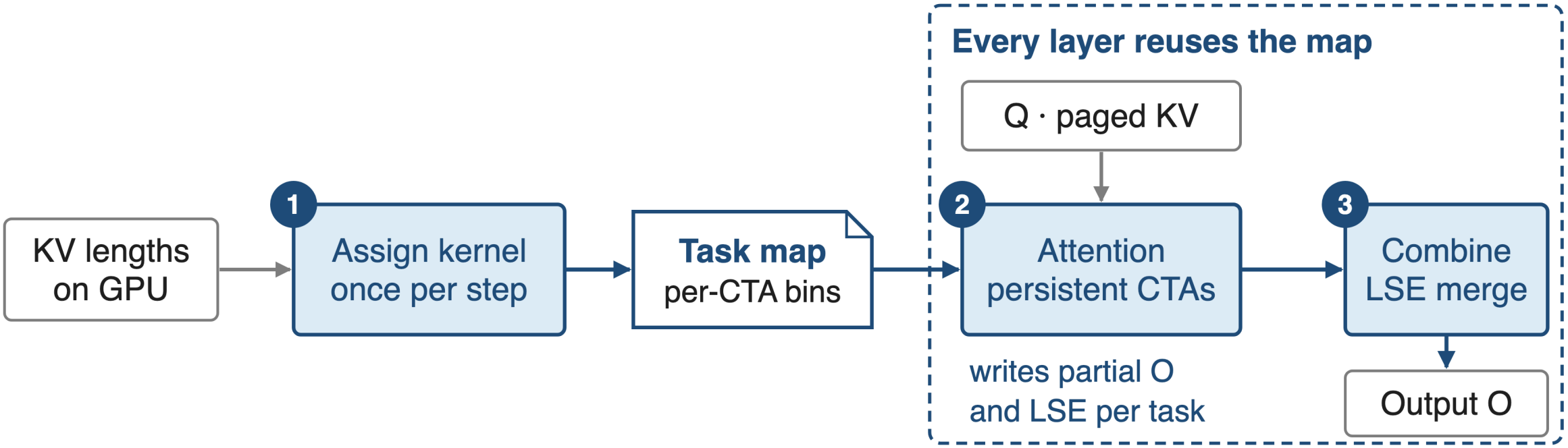
- Work = live KV in 64-token tiles; lengths already sit on the GPU
- Budget = total tiles ÷ CTAs gives every CTA equal work
- Uniform batch: both schedules tie at **1.00×**

Insight: balance KV tiles, not requests

- Work = live KV in 64-token tiles; lengths already sit on the GPU
- Budget = total tiles ÷ CTAs gives every CTA equal work
- Uniform batch: both schedules tie at **1.00×**

Decide the split every decode step from live lengths, not per request from a fixed policy

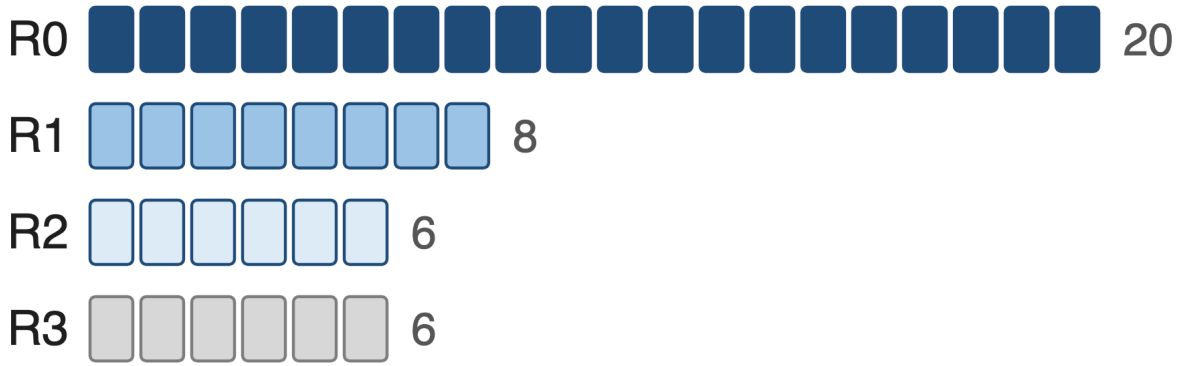
One task map per step drives every layer



Built on the GPU from device-side lengths; attention and combine launch with PDL

❶ The assign kernel sets a per-CTA tile budget

Live KV in 64-token tiles



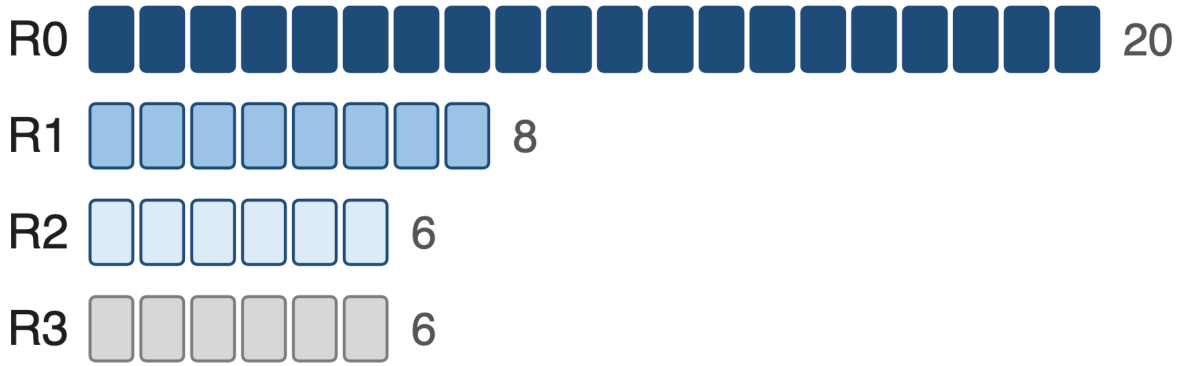
budget = ⌈40 tiles ÷ 4 CTAs⌉ = 10 tiles

floor: 512 tokens = 8 tiles per CTA

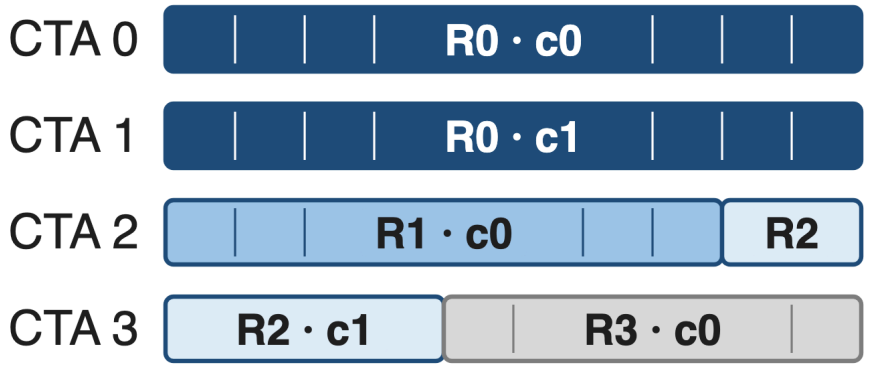
Toy batch with one KV head; the real budget counts tiles over all KV heads and requests

❷ Equal bins: long requests span CTAs

Live KV in 64-token tiles



Equal bins, one per CTA



budget = ⌈40 tiles ÷ 4 CTAs⌉ = 10 tiles

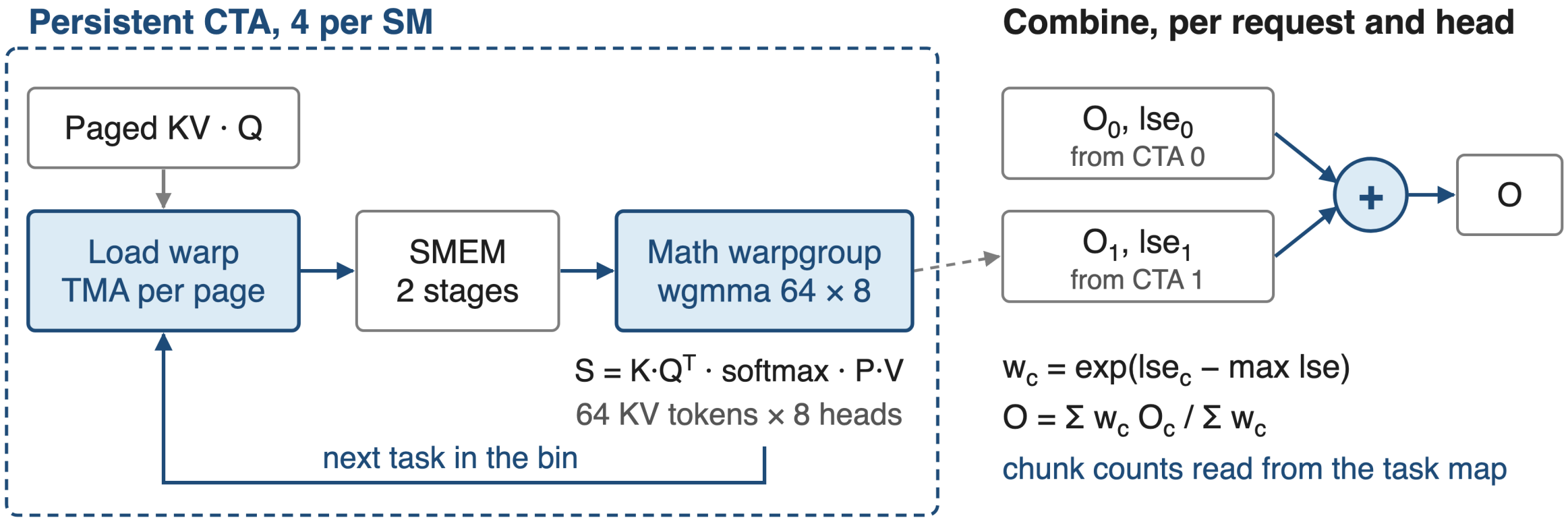
floor: 512 tokens = 8 tiles per CTA

task = (head, request, chunk, start, len)

CTA 1 → h0 · R0 · c1 · tokens 640–1279

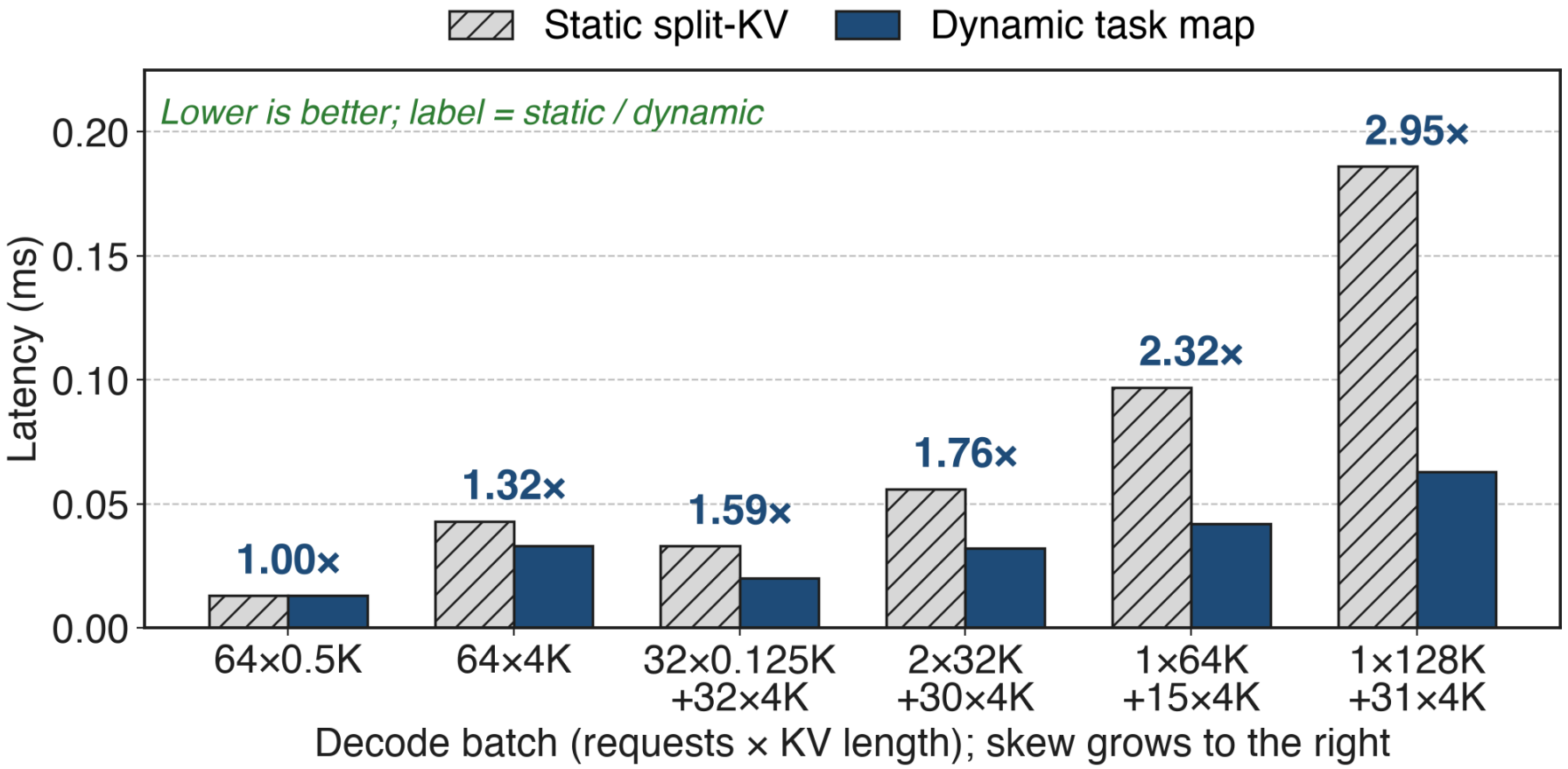
Fill order: KV head, then request; leftover tiles of a request spill into the next bin

❸ CTAs drain bins; combine merges by LSE



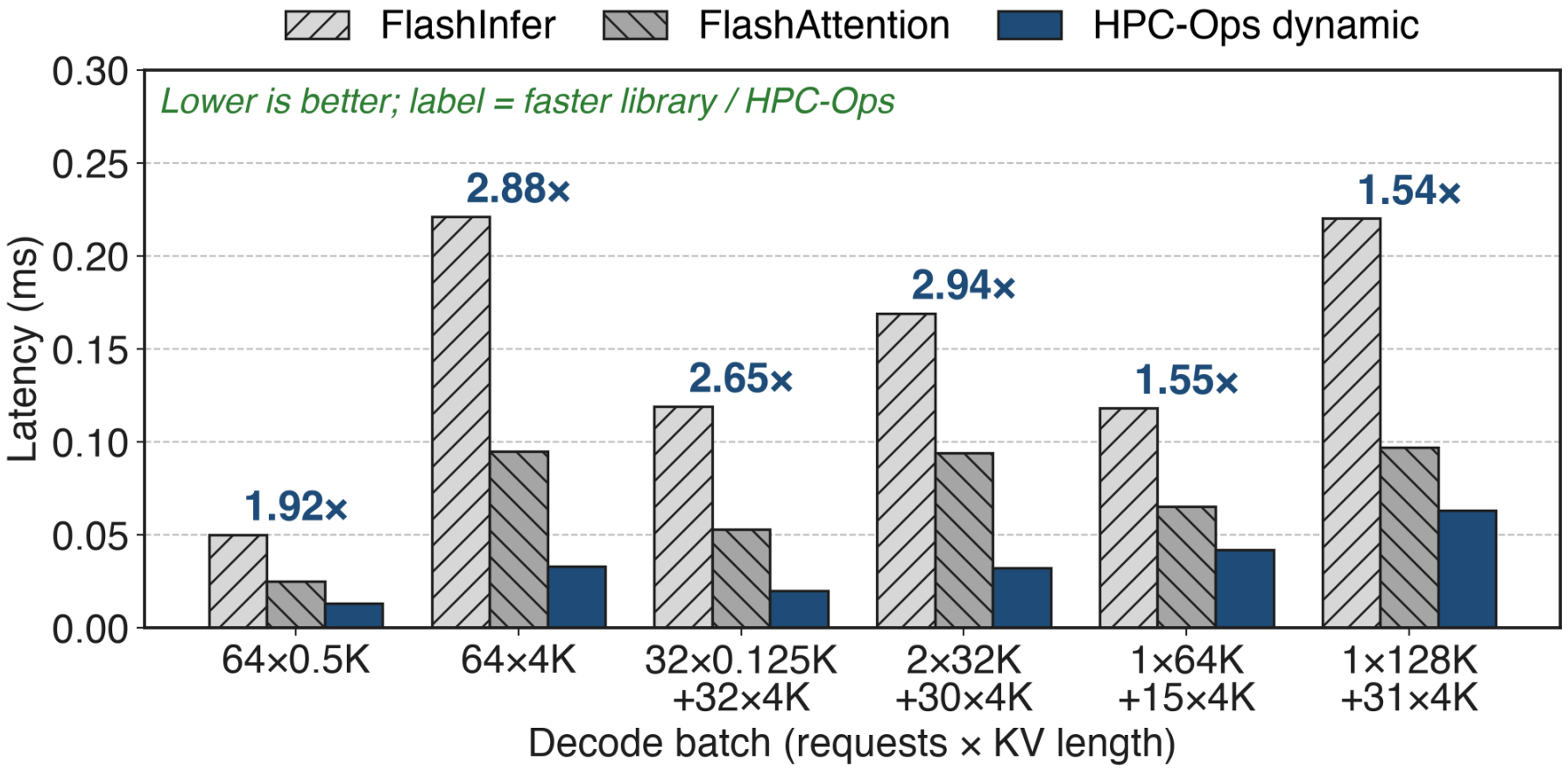
Combine switches to a multi-warp path above 24 chunks per request and head

Gains grow with skew, up to 2.95×



H20, FP8 KV-cache decode; HPC-Ops static split-KV vs dynamic task map (LMSYS Table 1)

2.25× faster than the faster library on average



Labels: faster of FlashInfer / FlashAttention ÷ HPC-Ops dynamic per case (derived); blog mean 2.25×

End to end: modest alone, large with fused MoE

- H200 Hy3-FP8, vs static: output throughput **+2.0%**, TTFT **–5.3%**
- H200 Qwen3 BF16, vs static: throughput **+3.0%**, TPOT **–2.8%**
- 8 × H20 Hy3-FP8, with fused MoE: TPOT **–15.1% to –48.8%**

Alone, a 2–3× faster attention kernel is a few percent of throughput

Costs: one more kernel, SM90 only

- **Per step**: an assign kernel and workspace; the benchmark times it separately
- **Scope**: SM90, head_dim 128, GQA groups of 4 or 8, few MTP draft tokens
- **Residual skew**: 1×128K + 31×4K still **1.9×** slower than 64×4K

Summary

- Static split-KV: same KV work, **4.3×** the latency on a skewed batch
- **Insight**: balance 64-token KV tiles, not requests, every decode step
- Up to **2.95×** vs static; **2.25×** vs the faster library on average (H20)
- Library context: deck 26 (HPC-Ops); sparse prefill: deck 09

https://github.com/Tencent/hpc-ops